

Executive Summary

The **Financial Transactions Dataset: Analytics** (CaixaBank Tech, 2024) is a large synthetic credit-card transaction dataset designed for an AI hackathon [7†L248-L253] [28†L344-L353]. It contains ~13.3 million transactions (2010s), user demographics (income, debt, credit score, etc.), card details (brand, credit limit, type), merchant category codes (MCC), and a separate fraud-label file. Its rich schema and multi-year span support behavioral analysis (e.g. seasonal trends in transaction volume [28†L369-L374]) and linking transactions to customer profiles. In contrast to PaySim (a much smaller, mobile-money simulation from 2016 [81†L201-L209]), the CaixaBank data is **more feature-rich and realistic** for modern credit-card fraud modeling. Notably, it *avoids* the PaySim-style leakage from “balance” fields that plagued V1 [45†L581-L589].

However, the dataset has limitations: being synthetic, it may not capture all real-world fraud scenarios (no device/IP data, no inter-account transfers, limited merchant info). There is **no documented built-in leakage**, but caution is needed in feature engineering (see below). The data *can* support analyst tasks (it links evidence across transactions, users, cards and MCC categories), but lacks operational attributes like transaction location or dynamic balance.

Recommendation: Replace PaySim with the CaixaBank dataset for Version 2. It offers far richer, more realistic fraud scenarios and avoids known PaySim pitfalls (balance leakage). For demonstration purposes, PaySim can be retained, but modeling and evaluation should use the CaixaBank data. This recommendation is supported by the dataset’s provenance and structure [7†L248-L253] [28†L344-L353], and by the clear advantages over PaySim (see Section 9).

Dataset Provenance

The dataset was created by **CaixaBank Tech** (in collaboration with Nuwe) for the *2024 CaixaBank AI Hackathon* [7†L248-L253] [28†L344-L353]. It is intended for financial analysis, fraud detection, and related analytics challenges. The Kaggle description indicates it was “designed for multiple analytical purposes, including synthetic fraud detection, customer behavior analysis, and expense forecasting.” No official peer-reviewed publication has been found beyond hackathon notes and Kaggle. The data is **synthetic** – generated to resemble a banking institution’s records – though exact generation methods are undisclosed.

Licensing is **Apache 2.0** (per Kaggle metadata), permitting free use and modification. The Kaggle page and related repositories serve as primary documentation. Jan Siczynski’s repository notes the data is ~365 MB (zipped) spanning the 2010s [28†L344-L353]. There is no evidence of conflicting versions; multiple independent teams (Kaggle notebooks, GitHub solutions) reference the same “transactions_data.csv”, “users_data.csv”, “cards_data.csv”, “mcc_codes.json”, and “train_fraud_labels.json” files [7†L248-L253] [28†L344-L353].

Summary: CaixaBank Tech (2024) – synthetic, Apache-licensed, for hackathon analytics tasks [7†L248-L253] [28†L344-L353] .

Dataset Structure

- **Files:** The dataset includes:
 - `transactions_data.csv` – detailed transaction records ($\approx 13,305,915$ rows, 12 columns, ~ 1.2 GB raw [18†L211-L215] [45†L581-L589]).
 - `users_data.csv` (aka `clients_data`) – customer demographic/financial profiles (age, income, debt, credit score, etc.). Exact row count is unspecified but presumably large (tens or hundreds of thousands of customers).
 - `cards_data.csv` – card details per user (`card_id`, brand, type, credit_limit, etc.; ~ 510 KB file [52†L1-L3] , so likely a few thousand cards).
 - `mcc_codes.json` – MCC lookup (~ 109 merchant category codes, ~ 6 KB) [45†L581-L589] .
 - `train_fraud_labels.json` – fraud labels: maps `transaction_id` to binary fraud flag (size ~ 100 MB) [45†L581-L589] .
- **Schema and Fields:**
 - **Transactions:** Each record has *transaction_id*, *client_id*, *card_id*, *amount*, *timestamp*, *merchant_id*, *mcc_code*, and flags like *use_chip*, *use_swipe*, *is_online*, plus possibly an *error* field. The pipeline documentation explicitly lists: “transaction ID, client ID, card ID, amount, merchant, MCC code, timestamps, errors, etc.” [45†L581-L589] . For example, a recent analysis notes “txn_hour” as a feature, implying time-of-day is present.
 - **Customers:** Fields include *user_id* (*client_id*), demographics (age or age-group), *years_until_retirement*, *annual_income*, *total_debt*, *credit_score*, and possibly *region/state* (the hackathon Q8 analysis segments “4 US geographic regions” [28†L437-L446]). Exact schema is undocumented but inference from queries suggests standard credit variables.
 - **Cards:** Fields include *card_id*, *client_id* (owner), *card_brand*, *card_type* (e.g. debit/credit), *credit_limit*, and possibly *expiry_date* or similar (grudloff’s code references “latest expiry date” [38†L288-L293]).
 - **Fraud Labels:** Provided separately. The JSON `train_fraud_labels.json` contains a dictionary of {*transaction_id*: *fraud_flag*}. A recommended procedure (Jan Siczynski) is to convert it to CSV by mapping each ID to Yes/No or 1/0 [28†L503-L512] . Only **training** labels are given. The dataset seems intended for training an ML model; no test labels are provided (test transactions would be unlabeled for scoring).
 - **Timestamps:** Full date-time stamps are present (at least to hour granularity). The data span “the 2010s decade” [28†L346-L353] , presumably covering 2010–2019.
 - **Identifiers:** Unique IDs tie tables together: *client_id* in transactions links to users; *card_id* links to cards (and also to user via card table); *transaction_id* indexes fraud labels. Merchant info is only via *merchant_id* and *mcc_code*.
 - **Transaction Types:** The dataset includes payment method flags (chip, swipe, online), and a general transaction *type* field (e.g. “Payment” vs “Transfer” – unclear if transfers exist). Hackathon analyses refer to “payment method (chip, swipe, online)” [28†L426-L434] .

- **Geography:** The only geo-like field is likely *region* or *state* in user data (since Q8 breaks down by “US regions” [28†L437-L446]). There is no explicit field for transaction location (no city/state per merchant provided in available docs).
- **Relationships:** One-to-many from user→cards, user→transactions (through cards or directly via `client_id`), and card→transactions. Merchant_id and MCC link to merchant categories.

Entity-Relationship: Users (clients) have Profiles (income, credit, etc.) and one or more Cards (each with limits/types). Each Transaction record references one User and one Card, and includes an MCC code for the Merchant. Fraud labels link 1-1 to transactions. There is no separate merchant master table beyond MCC mapping.

Behavioural Modelling Assessment

The data is **richly longitudinal**: multiple transactions per client and per card over a decade, enabling true behavioral modeling. For example:

- **Repeated Entities:** Each `client_id` appears in many transactions (users make multiple purchases over time). Each `card_id` is likewise reused. Merchant IDs recur (likely ~hundreds or thousands of distinct merchants). The presence of user and card histories is built-in, unlike PaySim which used abstract “agent” names.
- **Temporal Sequences:** Transactions include timestamps to at least hour resolution. Users have histories across months/years. This allows modeling of seasonality (indeed, one analysis found clear seasonality with summer peaks [28†L369-L374]) and trend drift. Data spans the full 2010s, so e.g. analysts computed 2010-specific stats (Top spenders in 2010 [28†L468-L479]).
- **Customer History:** Static attributes (income, credit score, debt) plus transaction history enable profiles. For each user, one can compute time-series features (e.g. velocity, typical spending categories).
- **Payment Patterns:** Flags for chip vs swipe vs online allow distinguishing in-person vs card-not-present behavior. One hackathon question directly analyzed “share of each payment method (chip, swipe, online)” [28†L425-L433] . A model can thus learn different behaviors for online fraud vs offline.
- **Relationship Modeling:** Cards are linked to users, so one can infer card-ownership networks. If a fraudster steals a card, their activity appears under that user’s profile. The dataset doesn’t explicitly include transfer beneficiaries (no account-to-account fields), so “beneficiary chains” are not modeled here.

Quantitative Indicators: The transactions file (~13.3M rows) is large enough that even with, say, 100K users, average ~133 transactions per user. This scale is suitable for robust longitudinal features. The SQL analysis (Jan Siczynski) implies multiple tables and joins, supporting broad user histories [28†L344-L353] . The robust nature of analyses (seasonality, credit-score impacts) suggests genuine behavioral variation.

In summary, **behavioral richness is high**. Customers and cards recur frequently, timestamps cover years, and user profiles enable sequence modeling. This far exceeds PaySim’s synthetic agent interactions, which were limited to one month of data [81†L201-L209] . CaixaBank’s dataset genuinely supports customer- and card-level temporal analytics (e.g. monthly aggregates, velocity, profile-based patterns).

Leakage Risk Assessment

No *explicit* leakage issues are documented for this dataset. Unlike PaySim (where “oldbalance” fields led to label leakage), this data lacks running balances. However, **potential risks** include:

- **Temporal Leakage:** If models or features aggregate across time incorrectly, future data can leak into training. For example, one team discovered that computing a customer’s “home zip code” using *all* transactions (including future ones) caused a huge leakage and an unrealistic performance spike [41†L441-L449] . In our context, any cross-temporal aggregation (e.g. customer spending averages) must be computed on past data only.
- **Engineered Leakage:** Similar to above, aggregating fraud labels or outcomes into features (e.g. flagging customers with past fraud) would leak target info.
- **Proxy Variables:** Static user attributes (credit score, income) strongly predict fraud rate [28†L392-L400] . While not “leakage” per se, models might overly rely on them, conflating financial profile with fraud propensity. This may harm generalization.
- **No Simulator Artifacts:** There is no indication of glaring synthetic artifacts like PaySim’s balance-derivation. The Kaggle description even markets “synthetic fraud detection” but no leaked secrets.
- **Data Split Concerns:** Only training labels are provided. Any train/test split must ensure no temporal overlap. If using a random split, one risks training on future transactions of the same user. A time-based split (train on early years, test on later) is safer.

Evidence: The only documented “leakage” comes from downstream analysis [41†L441-L449] , not from the data itself. That incident highlights the danger of including future info in features. But no inherent feature in the dataset *directly* reveals fraud. In short, there is *no known dataset-originated target leakage*. Still, rigorous temporal validation is essential.

Operational Suitability Assessment

The dataset is **semi-suited for analyst workflows**, but with caveats:

- **Transaction Triage & Evidence:** Each transaction record is accompanied by contextual info: the client’s profile (income, debt, credit), card details, and merchant category. An analyst could, in principle, assemble a brief on a flagged transaction using these fields. For example, one could compare the transaction amount to the user’s income/debt or credit limit. The MCC code allows categorizing the merchant (e.g. “electronics store”), which can feed explanations. The presence of *timestamp*, *card usage type*, and *regional data* also aids profiling (though actual location isn’t given, region acts as a proxy).
- **Investigations & Explanations:** The data lacks narrative features (no text fields, merchant names, or locations), but numeric attributes can support rule-based explanations (“Transaction flagged because it’s high relative to usual income” or “online purchase in a high-risk MCC category”). Indeed, hackathon analyses used credit score and spending to infer risk [28†L392-L400] . However, without merchant descriptions or device/IP info, analysts’ reasoning is limited to statistical anomalies.
- **Recommendations & Decisions:** The dataset provides no explicit risk scores or action labels. A model trained on it could output a fraud probability, but recommending a human decision (investigate vs clear) is beyond the data. Nevertheless, features like credit utilization or sudden card usage could trigger automated advice (e.g. “increase monitoring”). The dataset itself does not

contain cost/fraud severity fields or labels of “appropriate action,” so recommendation logic would be user-defined.

- **Human-in-the-Loop:** The schema (customers, cards, MCC) is amenable to explainability: features are interpretable (income, debt, MCC category, etc.), and an analyst could query the history of a client or card. All key values (IDs, amounts, times) are auditable and reproducible from raw data.
- **Auditability:** Since it is synthetic data, audit trails can be constructed by design. All records have stable IDs; nothing “black box” or aggregated beyond basic stats.

Overall, the dataset is **better than PaySim** for decision support. PaySim only provides anonymized transaction flows with minimal context, whereas this data links to user/card profiles. It can support a simplified analyst workflow of “flag transaction → view user/card history → explain via features,” but it is still primarily a ML classification dataset. In particular, it lacks operational metadata (no customer accounts across banks, no CRM data, no communication logs), which limits its realism as a full investigation suite.

Feature Engineering Assessment

Possible Derived Features (supportable):

- **Account/Customer Lifetime:** With timestamps, one can compute “client tenure” as time since first transaction seen. (Exact account opening date is absent, but first-data-appearance is a proxy.)
- **Spending Velocity:** Inter-purchase time, transactions per day/week per user or card.
- **Monetary Aggregates:** For each user: total/average transaction amounts, variance, spending by MCC category. For each card: utilization ratio (transaction amount vs credit limit), recency of use.
- **Behavioral Baselines:** Per-user average spend per merchant category or per time-of-day, and deviation of each transaction from that baseline.
- **New vs Repeat:** “Merchant familiarity” – e.g. count of prior transactions by the user with the same merchant or MCC. “New beneficiary” concept doesn’t apply (no beneficiary field).
- **Rolling Averages:** 30-day or 90-day rolling sum of spend for each user.
- **Card Features:** Card age (since first transaction), number of cards per user.
- **Channel Usage:** For each user, frequency of online vs offline transactions.
- **Velocity Metrics:** Number of transactions in past hour/day/week.
- **Credit Features:** Ratio of transaction amount to credit limit or to annual income. Debt-to-income ratio (can use existing debt & income).

Missing/Unobtainable Features:

- **Device & Location:** No IP, device ID, or exact location data. Cannot tell if a transaction occurs in an unusual place.
- **Geographic Context:** Only broad region exists; exact merchant city or customer home address is missing.
- **Account Balance:** No checking account balance to compute credit utilization beyond card limit.
- **Payment Methods beyond Chip/Swipe/Online:** No e.g. EMV vs MagStripe details beyond the given flags.
- **External Networks:** No linkage to external fraud lists or blacklisted merchants.
- **Social/Network Features:** No data on customer’s connections, number of cards fraudulently used elsewhere, etc.

- **Prior Fraud History:** Without labels on past transactions (except within train set), one cannot directly compute “has this user had fraud before” unless making assumptions from training labels.

In summary, a wide range of operational features (temporal aggregates, ratios to income/limits, merchant-frequency metrics) can be engineered. The dataset supports many classic fraud features (velocity, rarity, user profile comparisons). Critical missing elements are device/geo context and account balances, which are often used in real fraud analytics but are absent here. This limits detection of certain fraud types (e.g. “sudden new location”) unless inferred from patterns in allowed fields.

Fraud Realism

The dataset simulates **credit-card-type fraud scenarios** rather than mobile-money or ACH fraud. Evidence of realism:

- **Card-Not-Present vs Present:** Flags for online transactions suggest inclusion of CNP scenarios, which dominate modern card fraud. The mix of chip/swipe vs online transactions allows simulating both in-person and online fraud.
- **Identity-Based Risk:** The analysis found poorer credit scores correlate with higher fraud rates [28†L392-L400] , a plausible real-world pattern (fraudulent users often have poor credit). This indicates the synthetic fraud injection aligned with known risk factors.
- **Card Features:** Card brand/type and credit limits are included. Some fraud may exploit high-limit cards. For example, an oversized purchase relative to limit/income can flag stolen card use.

Likely Represented Fraud Types:

- **Stolen/Lost Cards (Card Present):** High-value purchases or unusual patterns on a physically compromised card would be labeled. The chip/swipe flags permit modeling this.
- **Card-Not-Present Fraud:** Online purchases using stolen card details (shopping, e-commerce) are represented (the `is_online` flag exists). These are common in real fraud.
- **Account Takeover:** Not explicitly modeled; the data lacks a separate login or account ID beyond cards. If a fraudster took over an account, it would appear identically to a stolen-card scenario here.
- **Fraud Rings/Mules:** The dataset does not include inter-account transfers or multiple account linkages, so multi-leg laundering and mule networks are **not simulated**. All transactions are by known customers with known cards.
- **Layering/Social Engineering:** No fields indicate internal manipulation (like repeated small deposits/withdrawals). There is no explicit simulation of social engineering beyond the fraud flag itself.

Absent/Underrepresented Scenarios:

- **Transfers & Money Movement:** Because it is card-centric, standard money laundering schemes (moving funds via multiple accounts) are absent.
- **Emerging Payment Methods:** No e-wallet, mobile-wallet, or crypto-fraud events (all transactions are card-based).
- **Merchant Fraud:** There is no separate “merchant fraud” label. Malicious merchants (e.g. transaction laundering) aren't modeled.
- **Device/Geolocation Anomalies:** No ability to simulate traveling fraud (a transaction far from home location) since location is missing.

In summary, the fraud in this dataset is focused on realistic credit card misuse (card-present and card-not-present fraud). It captures many everyday fraud patterns through transaction amounts and user risk factors, but omits complex banking fraud schemas. Compared to **PaySim** (which simulated African mobile-money transactions in April 2015 [81†L201-L209]), this dataset better reflects modern card fraud with true user data. PaySim's fraud (mostly internal bank fraud scenarios) was quite different. Here, fraud realism is higher for credit card use, but the synthetic nature means some edge-case behaviors may be missing.

Responsible AI Assessment

Privacy: All data are synthetic/aggregate, with no real PII. Basic demographics (age, income, credit score) are privacy-sensitive, but since they're generated for a hackathon, re-identification risk is negligible. Apache-2.0 licensing imposes no additional privacy constraints beyond Kaggle's terms.

Representativeness & Bias: Being synthetic, population representativeness is unknown. If the generation process is not fully disclosed, the distributions (e.g. income, region, credit) may not match any real customer base. The user population appears U.S.-centric (with "4 US regions" [28†L437-L446]) – any model built may not generalize outside that context.

- **Demographic Bias:** Key protected attributes (race, gender) are not present (or not documented), reducing direct bias checks. However, credit score, income, debt and geography could correlate with real-world demographic groups. If, for instance, low-income or young users are overrepresented in fraud labels, a model might inadvertently discriminate. The hackathon analysis showing poor credit predicting fraud [28†L392-L400] suggests risk: models might learn "poor credit → fraudulent" which can compound socioeconomic bias if not handled.
- **Fairness:** Without demographic labels, standard fairness metrics (e.g. equalized odds by race) cannot be computed. One must assume and mitigate any correlation between fraud labels and client characteristics. For instance, using credit score or income features could entrench inequality if not carefully monitored. Synthetic data might hide subtle biases unless cross-validated with domain knowledge.

Governance & Transparency: The dataset's Apache license and Hackathon origin are clear, aiding governance. Feature definitions (amount, MCC, etc.) are standard and explainable. Since it's synthetic, concerns over data sovereignty or subject consent are moot. However, any model trained on it should note the **synthetic** nature: conclusions drawn may not hold on real data.

Synthetic Limitations: Synthetic fraud data often lack the full complexity of real networks. As one source notes, synthetic data can inadvertently encode biases from the generation model [78†L1-L8] . Here, the hackathon design likely used some internal logic to inject fraud – unknown to us – which could introduce artifacts. For example, if all fraud transactions happen in a narrow MCC set, a model might overfocus on those industries. Without real data validation, there's a risk of overfitting to these artifacts.

Explainability: On the positive side, the features are mostly tabular and interpretable. An analyst can readily understand "amount vs credit limit" or "credit score" in an explanation. The separate label file and clear schema help auditability.

Fairness Concerns: We must remain vigilant that models don't unfairly flag certain synthetic "customer segments" disproportionately. The absence of explicit sensitive attributes means fairness must be enforced indirectly, e.g. ensuring no group (by income bracket or region) is unduly targeted.

Comparison with PaySim

Aspect	CaixaBank Dataset (2024)	PaySim (2016) 【81†L201-L209】
Behavioral Realism	High: Realistic card transactions, multi-year span, true user/card links 【28†L344-L353】 【45†L581-L589】 . Supports longitudinal features and seasonal trends 【28†L369-L374】 .	Low: Synthetic mobile-money data limited to one month, agent-based with simplified behaviors 【81†L201-L209】 . Doesn't capture long-term customer behavior.
Leakage Risk	Low: No obvious feature leakage built-in. Must guard against temporal aggregation (like zip code leak 【41†L441-L449】).	High: Known "balance" fields allow easy label reconstruction (false leak) 【45†L581-L589】 . Other simulation artifacts exist (e.g. fixed network patterns).
Feature Richness	Very rich: Includes user demographics (income, debt, credit_score), card features (limit, type), timestamps, payment channel, MCC, etc. 【28†L344-L353】 【45†L581-L589】 .	Minimal: Only transactions with basic flags. No customer profiles, no external features. Primarily amount, old/new balances, type, merchant IDs.
Fraud Realism	Moderate: Emulates modern card fraud (CNP and POS). Includes correlation with risk factors (e.g. credit score) 【28†L392-L400】 . Lacks complex fraud chains (no money-laundering flows).	Limited: Simulates generic fraud on mobile wallets (agent-based theft), not directly comparable. No card data context.
Operational Suitability	Good: Has contextual data enabling evidence assembly (profiles, MCC). Still lacks some real-world details (device, location). Far more analyst-friendly than PaySim.	Poor: Only anonymized transaction flows. Hard to construct any human-readable "story". Not suited to decision support beyond toy classification.
Explainability	Good: Numeric features are intuitive (income, credit, MCC). Easy to justify alerts in financial terms.	Weak: Uses opaque agent names, balances. Features like "oldBalanceDest" are not intuitive in an audit.
Recommendation Support	Possible: Enough context to suggest actions (e.g. "this seems out-of-pattern for this customer"). No built-in labels for actions, but data can inform rules.	None: Data is just transactions. No user context to justify automated advice.

Aspect	CaixaBank Dataset (2024)	PaySim (2016) [81†L201-L209]
Data Prep Effort	Moderate: Large size (~1.2 GB CSV), multiple tables to join. Schema is well-documented (via hackathon/readme).	Easy: Smaller (few hundred MB), single merged CSV in Kaggle. Fewer joins.
Community Adoption	Emerging: Very new (2024) hackathon data. Limited publications so far.	Extensive: Widely used in research/competitions (often scaled-down). Many benchmark papers.

Key Improvements (CaixaBank vs PaySim): The CaixaBank dataset provides **orders of magnitude more realism and features**. It directly addresses PaySim’s weaknesses: no running balances to cause leakage; real multi-attribute customer and card data; and long-term time coverage. PaySim was designed for simulation, whereas CaixaBank’s data is closer to production banking logs. On the other hand, PaySim remains more battle-tested in literature. But given our need for *governance-first fraud support*, CaixaBank’s strengths (behavioral depth and explainability) outweigh PaySim’s traditional popularity.

Risks

1. Data Leakage (Moderate): As noted, indirect leakage can occur during feature engineering. A cautionary example is **Zip Code leakage**: one project inadvertently computed each customer’s “home zip” using all transactions, which included future data, causing a huge performance jump [41†L441-L449]. Mitigation: enforce strict training/test splits and compute aggregates only on past data.

2. Synthetic Artifacts (High): The data’s synthetic nature may hide unrealistic patterns. For example, fraud might only occur at specific MCCs or on specific days. Models might pick up such quirks that don’t generalize to real operations. Without validation on real data, there is a risk of **overfitting to simulation bias**.

3. Incomplete Scenarios (Moderate): Important fraud modes are missing (e.g. no device/IP, no cross-account transfers). A model might fail on real fraud involving these factors. This limits applicability to only certain types of fraud detection.

4. Feature Gaps (Moderate): Missing fields (location, transaction narrative) constrain explainability and evidence collection. Analysts may need to augment with other data sources in practice.

5. Imbalanced Data (Low/Moderate): While typical in fraud datasets, extreme imbalance can cause learning issues. (The dataset has ~0.15% fraud rate [41†L442-L450].) Careful sampling or specialized algorithms are needed.

6. Legal/Governance (Low): Apache 2.0 is permissive, but internal policy might require review of synthetic data use. No personal data removes privacy/legal hurdles.

7. Scalability (Low): The raw transactions file is large (~1.2 GB; 13M rows). This requires distributed or efficient processing (which Version 1 already handled). However, it is manageable with modern tools (the medium post reports loading 13M rows into RAM via Polars [18†L211-L215]).

Risks are ranked primarily by data integrity impact. The greatest concerns are synthetic biases and leakage in feature engineering (risks 1–2).

Final Recommendation

Replace PaySim completely with the CaixaBank dataset. Evidence strongly favors CaixaBank’s data for Version 2: it is far more realistic and feature-rich for fraud analysis [28†L344-L353] [45†L581-L589] and avoids the simulator-induced leakage that doomed PaySim-based models in V1. While PaySim can still serve as a demonstration example (due to its familiarity), all serious modeling and evaluation should use the CaixaBank dataset. It better supports behavioral modeling, explanation, and decision-support needs of our platform. This recommendation is supported by: the detailed schema and hackathon documentation [28†L344-L353] [45†L581-L589] , improved realism (see comparative analysis), and the positive absence of known data flaws.
